The Foundation

0%

Module 1: QUICK START (...

1.1 What You Need (5 Min Set...

1.2 Your First Folder

1.3 How to Structure Any Pro...

Module 2: WHY THIS WO...

2.1 Video Text Guide (Series ...

2.2 One Line of Python Trigge...

2.3 How a 1953 Word Game E...

2.4 The Ladder That Explains ...

2.5 Clawdbot (Moltbot) Has 1...

2.6 Video as Code: My AI Ani...

2.7 From Nazi Psychology to ...

Module 3: FOLDER ARCHI...

2

3.2 Customizing for Your Use ...

3.3 Common Mistakes and H...

3.2 Customizing for Your Use Case

What You'll Get From This

You will see how the three-layer folder architecture changes shape for three different kinds of work. The layers stay the same. The names, the context files, and the routing all change. You will walk away knowing how to build yours.

The Lesson

One structure, every use case

In **Section 3.1** you saw the full architecture: a top-level identity file (CLAUDE.md), workspace-level context files, and skills or tools that plug in where needed. The example in the video used a fake project with a community workspace, a production workspace, and a writing room.

That example was intentional. It is close enough to real work that you can see the logic, but generic enough that you have to make it your own. This lesson shows you how.

The principle is short: the layers do not change. The labels do. Your CLAUDE.md still sits at the top and routes everything. Each workspace still has its own context file. Skills still plug in where

Here are three real examples. Find the one closest to your situation, study it, then build your own.

Example 1: Content Creator

You make videos, write posts, manage a social presence, and probably do it mostly alone or with a very small team. Your work moves through a cycle: come up with ideas, write scripts or outlines, produce the content, publish it.

Your workspaces:

```
my-content-project/
├── CLAUDE.md
├── script-lab/
│   ├── CONTEXT.md
│   ├── ideas/
│   ├── drafts/
│   └── final/
├── production/
│   ├── CONTEXT.md
│   ├── briefs/
│   ├── specs/
│   ├── builds/
│   └── output/
└── distribution/
    ├── CONTEXT.md
    ├── platforms/
    ├── scheduling/
    └── analytics/
```

What each workspace does:

🟠 **Script Lab** — This is where thinking happens. Ideas go in. Drafts come out. The CONTEXT.md in this folder describes your voice, your audience, the kind of content you make, and the process you follow from idea to finished script. (could also

to, that goes in here. Claude reads this context and writes in your voice from the first line.

🟡 **Production** — This is where content gets built. If you are making animations (like the pipeline in **Section 2.6**), this is where your briefs, specs, and build files live. If you are making

simple videos, this might be where your shot lists, thumbnails, and description templates go. The CONTEXT.md here describes your production process, your tools, your visual standards.

🟢 **Distribution** — This is where finished content goes out. Platform-specific formatting (what works on Instagram vs LinkedIn vs YouTube), scheduling, repurposing long content into short clips. The CONTEXT.md describes your platforms, posting cadence, and any rules about how content should be adapted per channel.

What the CLAUDE.md looks like:

Your top-level file tells Claude what this project is and how to route between the three workspaces. Something like:

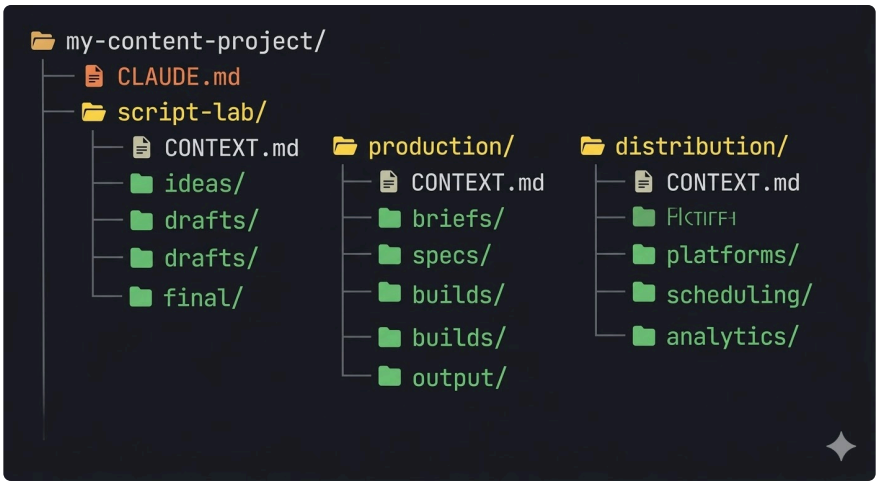
```
# My Content Project

I create [TYPE OF CONTENT] for [AUDIENCE].

## Workspaces
- /script-lab — Idea development, writing, drafts
- /production — Building and producing content
- /distribution — Publishing, scheduling, repurposing

## Routing
| Task | Go to | Read |
|------|-------|------|
| Write or brainstorm | /script-lab | CONTEXT.md |
| Build or produce | /production | CONTEXT.md |
| Publish or repurpose | /distribution | CONTEXT.md |

## Naming conventions
- Drafts: topic-name_draft.md
- Final scripts: topic-name_final.md
```



Example 2: Freelancer / Consultant

You work with multiple clients. Each engagement has a lifecycle: intake, scoping, delivery, follow-up. You need Claude to shift between clients without bleeding context. You also need your own internal workspace for business development, templates, and admin.

Your workspaces:

```
my-consulting-practice/
├── CLAUDE.md
├── client-alpha/
│   ├── CONTEXT.md
│   ├── intake/
│   ├── deliverables/
│   └── communications/
├── client-beta/
│   ├── CONTEXT.md
│   ├── intake/
│   ├── deliverables/
```



```
│   ├── CONTEXT.md
│   ├── proposals/
│   ├── reports/
│   └── frameworks/
└── business-dev/
```

```
├── CONTEXT.md
├── pipeline/
├── outreach/
└── case-studies/
```

What each workspace does:

🟠 **Client workspaces** (one per client) — Each client gets their own folder with their own CONTEXT.md. That file describes who the client is, what the engagement is, what phase you are in, what the deliverables are, and any client-specific rules (tone of voice, terminology they use, things to avoid). When you tell Claude "let's work on Alpha," it reads Alpha's context and nothing from Beta. No bleed. No confusion.

🟡 **Templates** — Your reusable frameworks. Proposal templates, report structures, analysis frameworks. The CONTEXT.md here describes what each template is for and how to use it. When you start a new engagement, you pull from templates into the client folder and customize.

🟢 **Business Dev** — Pipeline tracking, outreach drafts, case studies from past work. The CONTEXT.md describes your ideal client, your services, your positioning. When you need Claude to help draft an outreach email or build a case study, it reads this context.

What the CLAUDE.md looks like:

```
# My Consulting Practice

I am a [TYPE] consultant working with [TYPES OF CLIENTS].

## Active Clients
```

```
## Internal
- /templates — Reusable proposals, reports, frameworks
- /business-dev — Pipeline, outreach, case studies

## Routing
```

```
| Task | Go to | Read |
|------|-------|------|
| Client work for Alpha | /client-alpha | CONTEXT.md |
| Client work for Beta | /client-beta | CONTEXT.md |
| Build a new proposal | /templates | CONTEXT.md, then clier
| Outreach or pipeline | /business-dev | CONTEXT.md |

## Rules
- Never reference one client's information in another client
- Proposals always start from /templates and get customized
- Deliverables go in /client-[name]/deliverables, drafts sta
```

The key for freelancers: the client folders multiply. When you onboard a new client, you copy the structure, write a new CONTEXT.md, and Claude is ready. The CLAUDE.md at the top gets one new line in the routing table. That is it.

```
my-consulting-practice/
├── CLAUDE.md
├── client-alpha/
│   ├── CONTEXT.md
│   ├── intake/
│   ├── deliverables/
│   └── communications/
├── client-beta/
│   ├── CONTEXT.md
│   ├── intake/
│   ├── deliverables/
│   └── communications/
├── templates/
│   ├── CONTEXT.md
│   ├── proposals/
│   ├── reports/
│   └── frameworks/
└── business-dev/
    ├── CONTEXT.md
    ├── pipeline/
    ├── outreach/
    └── case-studies/
```

Example 3: Developer

You build software. You might work on one project or several

folder structure. The difference here is that Claude reads it.

Your workspaces:

```
my-app/
├── CLAUDE.md
├── planning/
│   ├── CONTEXT.md
│   ├── specs/
│   ├── architecture/
│   └── decisions/
├── src/
│   ├── CONTEXT.md
│   ├── components/
│   ├── services/
│   ├── utils/
│   └── tests/
├── docs/
│   ├── CONTEXT.md
│   ├── api/
│   ├── guides/
│   └── changelog/
└── ops/
    ├── CONTEXT.md
    ├── deploy/
    ├── monitoring/
    └── scripts/
```

What each workspace does:

🟠 **Planning** — Specs, architecture decisions, design docs. The CONTEXT.md describes the app, the tech stack, the current priorities, and any architectural principles you follow. When you tell Claude to help you spec a new feature, it reads this context and works within your existing architecture.

🟡 **Src** — The actual codebase. The CONTEXT.md here

writes code that fits your codebase because it read the rules first.

🟢 **Docs** — API documentation, user guides, changelogs. The CONTEXT.md describes your documentation standards, the

audience for each type of doc, and how docs relate to the code.

🔵 **Ops** — Deployment, monitoring, operational scripts. The CONTEXT.md describes your infrastructure, deploy process, and any runbook conventions.

What the CLAUDE.md looks like:

```
# My App

[APP NAME] — [One sentence description]

## Tech Stack
- Frontend: [framework]
- Backend: [language/framework]
- Database: [type]
- Deploy: [platform]

## Workspaces
- /planning — Specs, architecture, decisions
- /src — Application code
- /docs — Documentation
- /ops — Deployment and operations

## Routing
| Task | Go to | Read | Skills |
|------|-------|------|--------|
| Spec a feature | /planning | CONTEXT.md | — |
| Write code | /src | CONTEXT.md | testing-skill |
| Write docs | /docs | CONTEXT.md | doc-authoring-skill |
| Deploy or debug | /ops | CONTEXT.md | — |

## Naming conventions
- Specs: feature-name_spec.md
- Components: PascalCase
```

Developers will notice the routing table has a Skills column. This is Layer 3 from 3.1. You can wire testing skills, documentation skills, or code review skills into specific workspaces so they

only load when relevant. Your planning workspace does not need the testing skill. Your src workspace does

```
my-app/
├── CLAUDE.md
├── planning/
│   ├── CONTEXT.md
│   ├── specs/
│   ├── architecture/
│   └── decisions/
├── src/
│   ├── CONTEXT.md
│   ├── components/
│   ├── services/
│   ├── utils/
│   └── tests/
├── docs/
│   ├── CONTEXT.md
│   ├── api/
│   ├── guides/
│   └── changelog/
└── ops/
    ├── CONTEXT.md
    ├── deploy/
    ├── monitoring/
    └── scripts/
```

How to build yours

You do not need to match any of these exactly. The process is the same regardless of what you do.

Step 1: List your workspaces. Think about the 2-4 major areas of your work. What are the modes you shift between? Writing and building are different workspaces. Client A and Client B are different workspaces. Planning and executing might be different workspaces. If you find yourself wishing Claude would "forget" what it was just doing and focus on something else, that is a workspace boundary.

Step 2: Write a CONTEXT.md for each one. Describe what happens in this workspace, what the process is, what files live here, and what good work looks like. Keep it under a page. You can always add more later.

Step 3: Write your CLAUDE.md. List the workspaces and build the routing table. For each type of task, tell Claude where to go

Step 4: Start working. Point Claude at the folder and give it a task. See what happens. Adjust the context files based on what Claude gets right and what it gets wrong. The first version will not be perfect. It will be better than no structure at all, and it will get better every time you edit a context file.

The context files are living documents. Edit them as your projects change, as you learn what Claude needs to know, as you figure out what to cut. The people who get the most out of this system are the ones who treat the context files like working notes, not finished documents.

Resources

- **Premium and VIP members:** the full **Workspace Blueprint** and Example Production MD files are in **The Vault**

Go Deeper

- If you have not done the full walkthrough yet, **3.1: The Full Walkthrough** covers the three-layer system, naming conventions, and the research behind all of it.
- Common mistakes people make when setting up their folders are covered in **3.3: Common Mistakes and How to Fix Them**. Worth reading before you finalize your structure.
- The prompting framework from **1.3 How to Structure Any Prompt** works inside each workspace. Your context files handle identity and project context. Your prompts handle task, constraints, and output format. The two systems work together.
- The production pipeline example from the **2.6: Video as Code** lesson is a real implementation of the content creator structure above. If you want to see what a mature version of this looks like in action, start there.

Next Up: 3.3 Common Mistakes and How to

Before you finalize your setup, take five minutes to read the mistakes people make most often. Easier to avoid them now than fix them later.

Matthew Creamer

24d • Lesson Check-Ins

Foundations 3.2 Check-In

You just saw how the folder structure adapts to different use cases. Vote below, then drop your customized setup in

Poll 119 members have voted

49 63 New comment 2h ago